

MarkIt



- [MarkIt](#)
 - [Unsere Idee](#)
 - [Must-Haves](#)
 - [Nice-To-Haves](#)
 - [Aufgabenteilung \(circa\)](#)
 - [Roadmap](#)
 - [Server-Funktionen](#)
 - [Skizzen](#)
 - [Klassen-UML](#)

Unsere Idee

MarkIt ist ein Texteditor für Markdown-Dateien. Als Grundprinzip orientieren wir uns an **MS Word**. Man kann ganz normal Dateien öffnen, schreiben und wieder speichern. Man kann außerdem die **Basic-Markdown-Syntax** anwenden: Wenn man zum Beispiel auf einen Button klickt, wird der Text zu **`**bold**`**. Zu anderen Editoren unterscheidet sich unserer in einem wichtigen Punkt: JSON. Bei uns kann man mit einem Klick direkt das JSON-Format uebernehmen, also die {} oder [{}, {}].

Must-Haves

- UI wie ein Texteditor (mit einem Feld zum Schreiben, ...)
- Lesen von Markdown-, Text- und JSON-Dateien
- Schreiben von Markdown-, Text- und JSON-Dateien
- Basic-Settings (Farbschema, Bildgröße, Textgröße, Speicherpfade, ...)
- Datei-Verlauf: Welche Dateien wurden zuletzt geöffnet/verwendet?
- Real-Time-Rendering
- Verschiedene User-Accounts (Supabase)
- 2-Faktor-Authentifizierung (Supabase)

Nice-To-Haves

- Work-Together (Supabase)
- Bilder importieren (wie in VS Code)
- Cloud-Sync (Supabase)

Aufgabenteilung (circa)

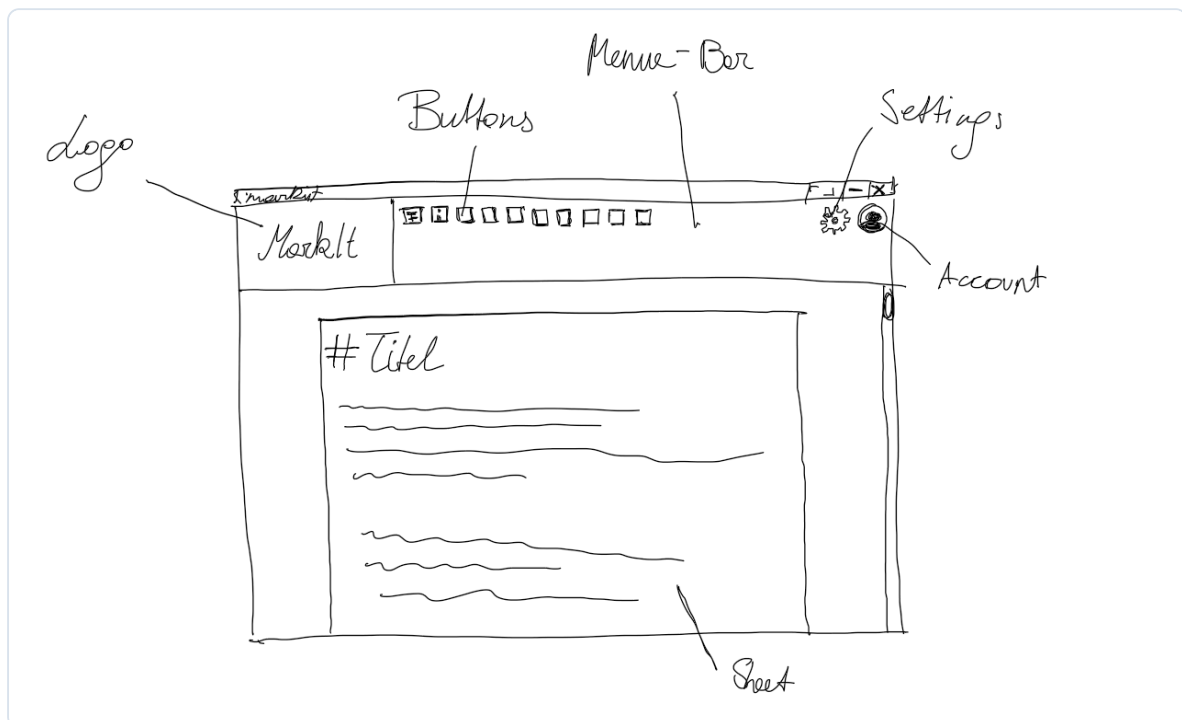
Karim	Max
Settings	Cloud-Sync
Lesen von Dateien	Dokumentation & Planung
Schreiben von Dateien	Datei-Verlauf
WPF-UI mit Backend	WPF-UI mit Backend
2-Faktor-Authentifizierung	User-Profile
Work-Together	Work-Together
Real-Time-Rendering	Real-Time-Rendering

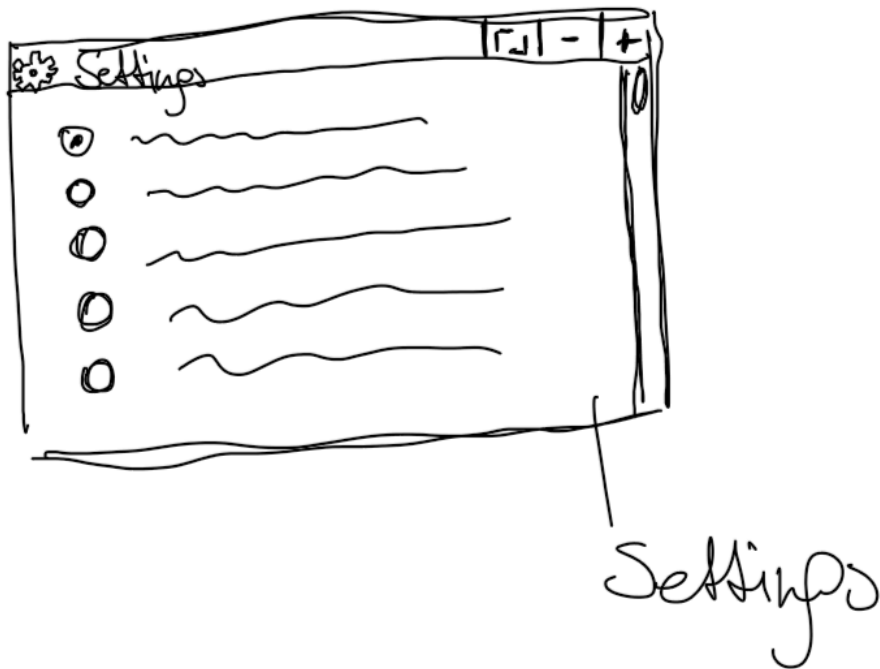
Roadmap

Server-Funktionen

Alle Server Sachen (Cloud-Sync, Userprofile, Work-Together...) machen wir alles ueber Supabase auf unserem eigenen Server. Supabase hat das ganze Backend fuer diese Funktionen schon. Wir haben nur Supabase installiert und aufgesetzt (Docker und die API).

Skizzen





LOGIN

E-Mail Address



Password



☐ Remember Me [Forgot Password](#)

LOGIN

[Register instead](#) [Continue as guest](#)

Klassen-UML

